

Chapitre 1 : Introduction

1. Génie logiciel et UML

Le génie logiciel (anglais software engineering) est une science de génie industriel qui étudie les méthodes de travail et les bonnes pratiques des ingénieurs qui développent des logiciels. Il a été défini du 7 au 11 octobre 1968 sous le parrainage de L'OTAN.

Le génie logiciel s'intéresse en particulier aux procédures systématiques qui permettent d'arriver à ce que des logiciels de grande taille correspondent aux attentes du client, soient fiables, aient un coût d'entretien réduit et de bonnes performances tout en respectant les délais et les coûts de construction.

Il existe maintenant des millions de pages sur le sujet GL, des centaines de conférences chaque année, plusieurs programmes universitaires et des millions de praticiens partout dans le monde

1.1 La crise du logiciel

A la fin des années 60, avec l'avènement des machines de 3^{ème} génération est apparu la crise du logiciel. Ces machines rendaient possible des applications jusqu'alors jamais réalisées. Les mécanismes utilisés pour les petits systèmes voyaient leur limite sur ces grosses applications. Les systèmes étaient peu fiables, difficiles à maintenir et avec de faible performance.

De nouvelles techniques et méthodes pour concevoir et maîtriser ces nouveaux logiciels devenaient indispensables. Il y avait donc une nécessité de passer d'une démarche artisanale à une discipline d'ingénieur.

Pour sortir de la crise, en 1968 sous le parrainage de l'OTAN, un groupe de chercheurs et de praticiens crée le génie logiciel (anglais, software engineering).

Le génie logiciel est défini comme un domaine qui couvre les méthodes, la modélisation, les techniques, les outils, les activités, les biens livrables et la gestion de projets relatifs au développement et à la maintenance du logiciel.

En s'appuyant sur une approche méthodique, l'équipe de développement accroît sa productivité et réalise des logiciels de qualité. De plus, il devient plus facile aux gestionnaires de « prédire » l'échéancier puisque les étapes sont connues.

Depuis 1968, on assiste à une prolifération et à une évolution continue :

- des méthodes d'analyse et des notations pour spécifier les fonctionnalités d'un logiciel;
- des techniques de conception et des notations pour exprimer la solution;
- des techniques et des langages de programmation;
- des procédures de validation et de vérification pour assurer la qualité du logiciel;
- des procédures de maintenance.

1.2 Définitions

Génie = science de l'ingénieur à l'instar des génies aéronautiques, maritime, civil...

Logiciel = objet manufacturé complexe avec ses propres techniques de fabrication

Selon l'ISO (organisation internationale de normalisation) :

« le génie logiciel est une création intellectuelle rassemblant des programmes, des procédures, des règles et de la documentation utilisée pour faire fonctionner un système informatique ».

Voici une autre définition :

« Le génie logiciel est un ensemble de procédures, de méthodes, de langages, d'ateliers, imposés ou préconisés par normes, adaptées à l'environnement d'utilisation afin de favoriser la production et la maintenance de composants logiciels de qualité »

En d'autres termes, le génie logiciel se base sur des principes, propose des méthodes et offre des techniques permettant de développer des logiciels de qualité, de façon économique et dans des délais fixés. De plus, le génie logiciel traite aussi bien des aspects d'organisation des équipes que des problèmes techniques liés au développement.

1.3 Objectifs et intérêts

Objectifs:

- Maîtrise des coûts, des délais, de la qualité.
- Etude des lois, méthodes suivies par le logiciel.

Intérêts:

- Prise en compte du coût exorbitant d'une erreur (humain ou financier).
- Prise en compte du cycle de vie.

1.4 Différents types du logiciel

- Logiciel système (utilitaires pour d'autres systèmes).
- Logiciel temps réels
- Logiciel de gestion (applications classiques).
- Logiciel scientifique.
- Logiciel critique.
- Logiciel combinatoire (IA).

Aujourd'hui, les gros logiciels représentent des millions de lignes de code. Réaliser et faire évoluer de tels logiciels reste encore un défi.

1.5 Questions généralement posés à propos du GL

Question	Réponse
Qu'est-ce qu'un logiciel ?	Les programmes informatiques et la documentation associée.

Quels sont les attributs d'un bon logiciel ?	Un bon logiciel doit offrir la fonctionnalité et les performances requises pour l'utilisateur et doit être maintenable, fiable et utilisable.
Qu'est-ce que le génie logiciel ?	Le génie logiciel est une discipline d'ingénierie qui s'occupe de tous les aspects de la production de logiciels.
Quelles sont les activités fondamentales du génie logiciel ?	Spécification du logiciel, développement du logiciel (conception et implémentation), validation du logiciel et évolution du logiciel.
Quels sont les principaux défis de l'ingénierie logicielle ?	Faire face à la diversité croissante, les demandes de délais de livraison réduits et le développement de logiciels dignes de confiance.
Quels sont les coûts du génie logiciel ?	Environ 60% des coûts liés aux logiciels sont les coûts de développement, 40% sont les coûts des tests.
Quelles sont les meilleures techniques et méthodes du génie logiciel ?	Bien que tous les projets logiciels doivent être gérés et développé par des professionnels, différentes techniques sont appropriées pour différents types de systèmes. Par exemple, les jeux doivent toujours être développés en utilisant une série de prototypes alors que les systèmes de contrôle critiques sécurisés exigent une spécification complète et analysable à développer. Par conséquent, on ne peut pas dire qu'une méthode est meilleure qu'une autre.
Quelles différences a réalisé le Web pour le génie logiciel ?	Le Web a conduit à la disponibilité de services logiciels et la possibilité de développer des systèmes à base de services hautement distribués. Le développement de systèmes basés sur le Web a conduit à d'importants progrès dans les langages de programmation et la réutilisation du logiciel.
Quels sont les coûts du génie logiciel ?	Environ 60% des coûts liés aux logiciels sont les coûts de développement, 40% sont les coûts des tests.

2. Modèles, modélisation et UML

2.1 Les modèles

- ✓ **Définition :**
 - Un modèle est une représentation abstraite de la réalité qui exclut certains détails du monde réel.
- ✓ **Utilité :**
 - Reflète ce que le concepteur croit important pour la compréhension et la prédiction du phénomène modélisé, les limites du phénomène modélisé dépendent des objectifs du modèle.
 - Permet de réduire la complexité d'un phénomène en éliminant les détails qui n'influencent pas son comportement de manière significative.
- ✓ **Exemple de modèles :**
 - Modèle météorologique : A partir de données d'observation (satellite...), il permet de prévoir les conditions climatiques pour les jours à venir.

2.2 La modélisation

A. Définition :

La modélisation est un processus par lequel on arrive à élaborer un modèle décrivant un système réel (ou un phénomène du monde réel).

B. Pourquoi modéliser ?

Modéliser un système avant sa réalisation permet de mieux comprendre le fonctionnement du système. C'est également un bon moyen de maîtriser sa complexité et d'assurer sa cohérence.

C. Types de modélisation :

✓ Modélisation à priori :

➤ Modéliser un système avant sa réalisation (le système n'existe pas encore).

➤ Objectifs :

- Comprendre le fonctionnement du futur système.
- Mesurer et Maîtriser sa complexité.
- Assurer sa cohérence.
- Pouvoir communiquer au sein de l'équipe de réalisation.

✓ Modélisation à posteriori :

➤ Modéliser un système après sa réalisation (le système existe déjà).

➤ Objectifs :

- Corriger les erreurs dans l'ancien système.
- Faire évoluer l'ancien système.

2.3 UML

A. Qu'est-ce qu'UML ?

Le langage de modélisation unifié (UML) a été créé comme un langage de modélisation visuelle commun, sémantiquement et syntaxiquement riche, pour l'architecture, la conception et la mise en œuvre de systèmes logiciels complexes, tant sur le plan structurel que comportemental. UML n'est pas un langage de programmation mais il existe des outils qui peuvent être utilisés pour générer du code dans différents langages à l'aide de diagrammes UML. UML a une relation directe avec l'analyse et la conception orientées objet.

B. Concepts de modélisation spécifiés par UML

Le développement du système se concentre sur trois modèles de système globalement différents :

✓ **Fonctionnel** : il s'agit de diagrammes de cas d'utilisation qui décrivent les fonctionnalités du système du point de vue de l'utilisateur.

✓ **Objet** : ce sont des diagrammes de classes qui décrivent la structure du système en termes d'objets, d'attributs, d'associations et d'opérations.

✓ **Dynamique** : les diagrammes d'interaction, les diagrammes de machines d'état et les diagrammes d'activité sont utilisés pour décrire le comportement interne du système.

Ces modèles de système sont visualisés à travers deux types différents de diagrammes : structurels et comportementaux.

3. Introduction à la modélisation Orientée Objet

3.1 Approches de modélisation pour le logiciel

A. Approche fonctionnelle :

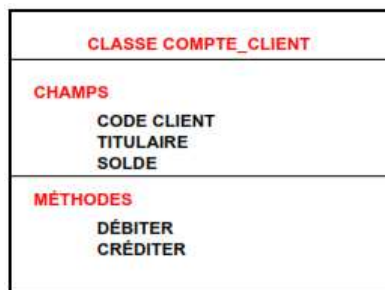
- Approche traditionnelle basée sur l'utilisation des procédures et des fonctions.
- Les grands programmes sont décomposés en sous-programmes.
- La modélisation fonctionnelle est une **approche descendante** :
 - La modélisation du système se base sur les fonctions, et non pas sur les objets.
 - On commence par déterminer la fonction globale du système.
 - Puis, on décompose la fonction globale du système en plusieurs sous-fonctions jusqu'à obtenir des fonctions élémentaires simples à programmer.
 - Il s'agit donc d'une démarche descendante.
- **Avantages :**
 - Adéquate pour les petits logiciels et les systèmes peu complexes.
 - Démarche ordonnée et organisée.
- **Inconvénients :**
 - Pose des problèmes de structuration de données, car elle est orientée fonctions.
 - Produit des logiciels non réutilisables.
 - Produit des logiciels très difficile à les faire évoluer ou corriger.

B. Approche orientée objets :

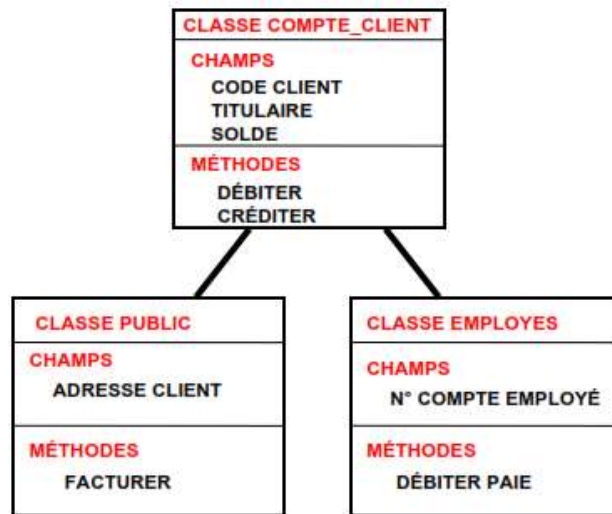
- On identifie les éléments du système et on en fait des objets.
- On cherche à faire collaborer ces objets pour qu'ils accomplissent la tâche voulue.
- C'est une approche fondée sur les objets :
 - Le modèle à produire est décrit en termes d'objets et non pas en termes de fonctions.
 - On peut partir des objets du domaine (briques de base) et remonter vers le système global.
 - On définit également les interactions et les collaborations entre les objets du système.
 - Il s'agit essentiellement d'une **approche ascendante**.
- **Avantages :**
 - Démarche naturelle et logique.
 - Raisonnement par abstraction sur les objets du domaine.
- **Inconvénients :**
 - Parfois moins intuitive que l'approche fonctionnelle.
 - Rien dans les concepts de base objets ne précise comment modéliser la structure objet d'un système de manière adéquate.
- **Différences entre COO (Conception orientée objet) & POO (Programmation orientée objet):**
 - La conception objet doit être dissociée de la programmation objet.
 - Les langages à objets ne présentent que des manières particulières d'implémenter certains concepts. Ils ne valident en rien la conception.
 - Il est même possible de concevoir objet et de coder en fonctionnelle (Langage C, par exemple).
 - Connaître des langages de programmation objet comme C++ ou Java n'est pas suffisant.
 - Il faut savoir s'en servir en utilisant un moyen de modélisation, d'analyse et de conception objet.
 - Sans les langages de haut niveau (donc de modélisation), il est difficile de comparer plusieurs solutions de modélisation objet.

3.2 Modélisation orientée objet

- **C'est une conception applicative**
 - les fonctions changent ; les objets restent
 - penser réutilisation
 - unité de conception = classe d'objets
 - langages: Java, C++
 - Donne une vue **"Sur quoi le système agit-il?"**
- **Un objet, c'est:**
 - Un ensemble d'attributs soumis à des actions spécifiques
 - les attributs -> état de l'objet
 - les actions -> les méthodes
 - Un objet a un comportement:
 - ✓ déterminé par ses actions
 - ✓ avec des contraintes sur les actions ou attributs
- **Classe d'objets**
 - ensemble d'objets
 - ✓ mêmes attributs statiques
 - ✓ mêmes attributs dynamiques
 - une instance est un objet particulier de la classe
 - ✓ crée avec les attributs d'états de la classe
 - ✓ se comporte selon les méthodes de la classe



- **Héritage**
 - ✓ des sous-classes: organisation hiérarchique
 - ✓ héritage des champs et des méthodes
 - extension des champs et méthodes de la classe dans la sous-classe
 - évite la duplication de code
 - ✓ héritages:
 - **simple**: nouvelle classe issue d'une classe origine
 - de toutes les propriétés
 - de certaines propriétés
 - **multiple**: nouvelle classe issue de plusieurs classes



- **L'encapsulation**
 - Opacité du fonctionnement interne d'un objet
 - Accès aux seules propriétés et services nécessaires
- **Le polymorphisme**
 - Traitement l'adaptant aux diverses versions des objets
- **La surcharge**
 - Réécriture d'un traitement hérité: ajout ou suppression de fonctionnalités
- **L'abstraction**
 - Le traitement (abstrait) est défini en fonction des objets inférieurs

4. Diagrammes de Cas d'utilisation

Les diagrammes de cas d'utilisation UML sont un outil précieux pour la modélisation des besoins fonctionnels d'un système logiciel et pour comprendre comment les acteurs interagissent avec ce système. Ils sont largement utilisés dans le processus de conception logicielle pour capturer les exigences et définir le comportement attendu du système. Ils sont donc utilisés pour modéliser les interactions entre les acteurs (utilisateurs) et un système logiciel.

L'ensemble des cas d'utilisation d'un système contient les exigences fonctionnelles attendues ou existantes, les acteurs (utilisateurs du système) ainsi que les relations qui unissent acteurs et fonctionnalités. Cet ensemble détermine également les frontières du système, à savoir les fonctionnalités remplies par le système et celles qui lui sont externes.

Les cas d'utilisation constituent un référentiel du dialogue entre les informaticiens et les clients et, par conséquent, une base pour l'élaboration au niveau fonctionnel du cahier des charges.

4.1 Éléments Clés

- **Acteur** : Un acteur est une entité externe au système qui interagit avec celui-ci. Les acteurs peuvent être des utilisateurs humains, d'autres systèmes, des périphériques matériels, etc.
- **Cas d'Utilisation** : Un cas d'utilisation représente une fonction ou un service spécifique offert par le système du point de vue de l'utilisateur. Il décrit ce que le système fait en réponse à une interaction de l'acteur.

- **Relation d'Association** : Une relation d'association connecte un acteur à un cas d'utilisation pour indiquer quel acteur participe à quelles fonctionnalités.

4.2 Notation UML pour les Diagrammes de Cas d'Utilisation

Les diagrammes de cas d'utilisation utilisent une notation spécifique pour représenter les éléments clés. Voici quelques symboles couramment utilisés :

4.2.1 Acteur :

Il est représenté par un petit bonhomme avec son nom (ou son rôle) inscrit dessous.



Il est également possible de représenter un acteur (non humain) sous la forme d'un petit rectangle.

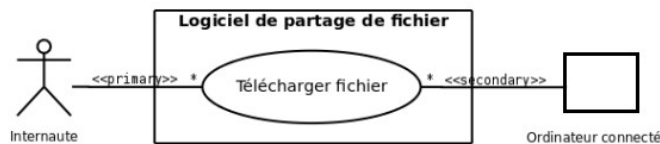
4.2.2 Cas d'Utilisation :

Un cas d'utilisation se représente par une ellipse contenant le nom du cas (un verbe à l'infinitif), et optionnellement, au-dessus du nom, un stéréotype.



4.2.3 Relation d'Association :

C'est le chemin de communication entre un acteur et un cas d'utilisation et est représenté par un trait continu.



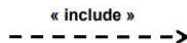
4.2.4 Les relations entre les cas d'utilisation

A- Inclusion :

La relation d'inclusion sert à enrichir un cas d'utilisation par un autre cas d'utilisation. Cet enrichissement est réalisé par une inclusion impérative, il est donc systématique.

L'inclusion sert à partager une fonctionnalité commune entre plusieurs cas d'utilisation. Elle peut également être employée pour structurer un cas d'utilisation en décrivant ses sous-fonctions.

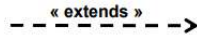
Cette relation est représentée sur un diagramme de cas d'utilisation par une flèche pointant vers un autre cas d'utilisation avec l'étiquette "<<include>>".



B- Extension :

Elle est utile pour représenter des fonctionnalités optionnelles qui ne sont pas nécessairement exécutées chaque fois que le cas d'utilisation principal est déclenché. La relation d'extension est principalement utilisée pour gérer les variations du comportement du système en fonction de certaines conditions ou d'événements particuliers.

Dans le diagramme des cas d'utilisation, cette relation est représentée par une flèche pointillée munie du stéréotype «extend».



C- La spécialisation et la généralisation des cas d'utilisation :

il est possible de spécialiser un cas d'utilisation en un autre. On obtient ainsi un sous-cas d'utilisation.

Le sous-cas hérite du comportement du sur-cas d'utilisation. Un sous-cas d'utilisation hérite également des relations de communication, d'inclusion et d'extension du sur-cas.

Dans le diagramme des cas d'utilisation, la relation de spécialisation est représentée par une flèche pleine de spécialisation identique à celle qui relie les sous-classes aux surclasses.



4.3 Étapes pour Créer un Diagramme de Cas d'Utilisation

La création d'un diagramme de cas d'utilisation UML implique généralement les étapes suivantes :

- **Identifier les Acteurs** : Identifiez les acteurs qui interagissent avec le système. Il peut s'agir d'utilisateurs humains, d'autres systèmes, ou de tout élément externe.
- **Identifier les Cas d'Utilisation** : Identifiez les fonctionnalités ou les services que le système doit offrir. Chaque cas d'utilisation représente une tâche spécifique que le système doit accomplir.
- **Établir les Relations** : Utilisez des relations d'association pour connecter les acteurs aux cas d'utilisation auxquels ils participent.
- **Définir les Inclusions et les Extensions (optionnel)** : Si certains cas d'utilisation sont inclus dans d'autres ou étendus par d'autres, utilisez des notations d'inclusion et d'extension pour les représenter.
- **Documenter les Cas d'Utilisation** : Pour chaque cas d'utilisation, créez une description détaillée qui explique son comportement, ses préconditions et ses postconditions.
- **Organiser le Diagramme** : Organisez les acteurs et les cas d'utilisation de manière logique sur le diagramme pour faciliter la compréhension.

4.4 Exemple

Définir un diagramme de cas d'utilisation UML décrivant la fonctionnalité d'une simple montre. L'utilisateur de la montre peut soit consulter l'heure sur sa montre ou régler l'heure. Toutefois, seul le réparateur de montres peut changer la pile de la montre.

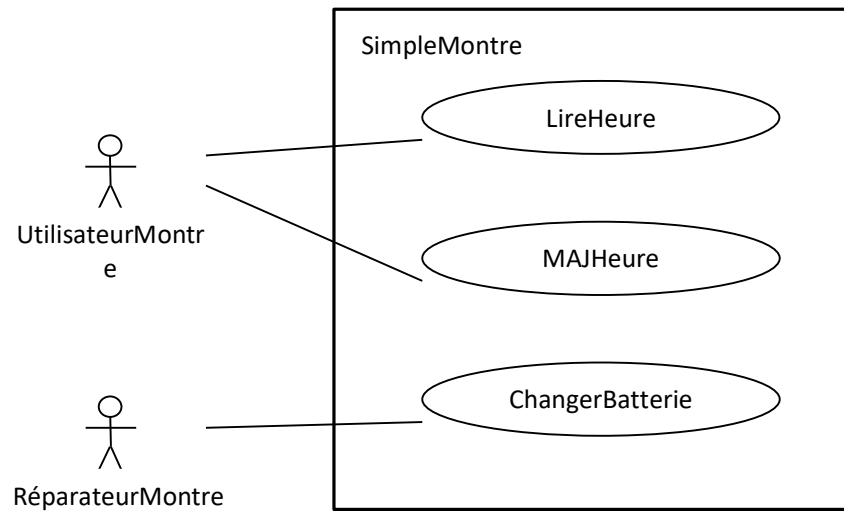


Diagramme des cas d'utilisation pour une simple montre